

DSpark: Confidence-Scheduled Speculative Decoding with Semi-Autoregressive Generation

DSpark: 基于半自回归生成的置信度调度推测解码

Xin Cheng, Xingkai Yu, Chenze Shao, Jiashi Li, Yunfan Xiong, et al.

北京大学、DeepSeek-AI

Abstract

摘要

Speculative decoding accelerates Large Language Model (LLM) inference by decoupling draft generation from target verification. While recent parallel drafters efficiently propose long token sequences in a single forward pass, they suffer from rapid acceptance decay due to a lack of inter-token dependencies. Furthermore, indiscriminately verifying these extended blocks wastes critical batch capacity on tokens with high rejection risks, severely degrading throughput in high-concurrency serving systems. We introduce DSpark, a speculative decoding framework that unifies high-throughput parallel generation with adaptive, load-aware verification. To maintain draft quality, DSpark utilizes a semi-autoregressive architecture—coupling a parallel backbone with a lightweight sequential module—to introduce intra-block dependency modeling and mitigate suffix decay. To optimize system efficiency, DSpark employs confidence-scheduled verification, dynamically tailoring the verification length for each request based on estimated prefix survival probabilities and engine-specific throughput profiles. On offline benchmarks across diverse domains, DSpark substantially improves the accepted length over state-of-the-art autoregressive and parallel drafters. When deployed within the DeepSeek-V4 serving system under live user traffic, DSpark successfully mitigates verification waste. Compared to the established production baseline (MTP-1), DSpark accelerates per-user generation speeds by 60%–85% at matched throughput levels. More importantly, by preventing severe throughput degradation under strict interactivity constraints, it enables performance tiers that were previously unattainable, shifting the Pareto frontier of our serving system. To facilitate community progress, we open-source the DSpark checkpoints alongside DeepSpec, an algorithm-driven training repository for speculative decoding.

推测解码通过将草稿生成与目标验证解耦来加速大语言模型（LLM）推理。虽然近期的并行草稿模型能在单次前向传播中高效生成成长token序列，但由于缺乏token间依赖关系，它们的接受率会快速衰减。此外，不加区分地验证这些扩展块会在高拒绝风险的token上浪费关键的批处理容量，严重降低高并发服务系统的吞吐量。本文提出DSpark，一个将高吞吐量并行生成与自适应、负载感知的验证相结合的推测解码框架。为保持草稿质量，DSpark采用半自回归架构——将并行主干与轻量级序列模块相结合——引入块内依赖建模并缓解后缀衰减问题。为优化系统效率，DSpark采用置信度调度验证，根据估计的前缀存活概率和引擎特定的吞吐量配置文件，为每个请求动态调整验证长度。在跨多个领域的离线基准测试中，DSpark相比最先进的自回归和并行草稿模型显著提高了接受长度。当在真实用户流量下部署于DeepSeek-V4服务系统时，DSpark成功减少了验证浪费。与已建立的生产基线（MTP-1）相比，在相同吞吐量水平下，DSpark将每用户生成速度提升了60%–85%。更重要的是，通过在严格的交互性约束下防止严重的吞吐量下降，它实现了以前无法达到的性能层级，推动了我们服务系统的帕累托前沿。为促进社区进步，我们开源了DSpark检查点以及DeepSpec，一个算法驱动的推测解码训练仓库。

1. Introduction

1. 引言

Large Language Models (LLMs) generate text autoregressively: each new token requires a full forward pass conditioned on all preceding tokens, making inference latency proportional to the output length. The resulting low GPU

utilization and high user-perceived waiting time constitute a primary bottleneck in production LLM serving, particularly for latency-sensitive scenarios such as real-time conversational assistants and multi-turn agentic workflows. Speculative decoding (Chen et al., 2023; Leviathan et al., 2023) offers a principled solution: a lightweight draft model proposes a block of candidate tokens, and the full-size target model verifies the entire block in a single forward pass via rejection sampling, accepting the longest prefix consistent with the target distribution and appending one bonus token. Because verification is parallel and the acceptance rule preserves the target distribution exactly, speculative decoding accelerates generation without any quality loss.

大语言模型（LLM）以自回归方式生成文本：每个新token都需要基于所有先前token进行完整的前向传播，使得推理延迟与输出长度成正比。由此导致的GPU利用率低和用户感知等待时间长，成为生产环境LLM服务的主要瓶颈，特别是对于延迟敏感的场景，如实时对话助手和多轮智能体工作流。推测解码（Chen等人，2023；Leviathan等人，2023）提供了一个有原则的解决方案：轻量级草稿模型提出候选token块，而完整的目标模型通过拒绝采样在单次前向传播中验证整个块，接受与目标分布一致的最长前缀并附加一个奖励token。由于验证是并行的，且接受规则精确保持目标分布，推测解码在不损失任何质量的情况下加速了生成。

The design of the draft model governs the trade-off between drafting latency and acceptance rate. Early drafters are autoregressive (Cheng et al., 2024; Li et al., 2024b), conditioning each position on previously sampled tokens. However, their drafting latency grows linearly with the block size, forcing these methods to use short blocks and shallow architectures. To break this sequential bottleneck, parallel drafters (Cai et al., 2024; Chen et al., 2026; Liu et al., 2026a) have emerged as a compelling alternative: all draft positions are produced in a single forward pass, making drafting latency nearly independent of block size. This structural advantage theoretically allows parallel drafters to efficiently generate substantially longer draft blocks.

草稿模型的设计决定了草稿延迟与接受率之间的权衡。早期的草稿模型是自回归的（Cheng等人，2024；Li等人，2024b），每个位置都基于先前采样的token进行条件生成。然而，它们的草稿延迟随块大小线性增长，迫使这些方法使用短块和浅层架构。为打破这一序列瓶颈，并行草稿模型（Cai等人，2024；Chen等人，2026；Liu等人，2026a）成为一个有吸引力的替代方案：所有草稿位置在单次前向传播中生成，使得草稿延迟几乎与块大小无关。这种结构优势理论上允许并行草稿模型高效生成更长的草稿块。

However, fully unlocking the potential of large parallel draft blocks introduces two critical bottlenecks—one in generation quality, and the other in system efficiency. First, because parallel drafters predict each position independently, they cannot model inter-token dependencies within a block. This independence leads to multi-modal collisions and rapid acceptance decay at later positions (Gu et al., 2018; Huang et al., 2022b). Second, determining the optimal verification length remains a challenge. While parallel generation easily produces long draft blocks, indiscriminately verifying all proposed tokens degrades system throughput, particularly under high-concurrency workloads (Hu et al., 2026; Liu et al., 2024c). The ideal verification length varies along two axes. On the data side, structured requests like code naturally sustain higher acceptance rates than open-ended chat (Abramovich et al., 2026; Xia et al., 2024). On the system side, verifying extra tokens is nearly free under light loads. Under heavy loads, however, verifying tokens with a high rejection risk occupies critical batch capacity that could otherwise serve other active requests (Liu et al., 2024b; Wu et al., 2025).

然而，充分释放大型并行草稿块的潜力会带来两个关键瓶颈——一个在生成质量方面，另一个在系统效率方面。首先，由于并行草稿模型独立预测每个位置，它们无法对块内的token间依赖关系建模。这种独立性导致多模态碰撞和后续位置的快速接受率衰减（Gu等人，2018；Huang等人，2022b）。其次，确定最佳验证长度仍然是一个挑战。虽然并行生成很容易产生长草稿块，但不加区分地验证所有提出的token会降低系统吞吐量，特别是在高并发工作负载下（Hu等人，2026；Liu等人，2024c）。理想的验证长度在两个维度上变化。在数据方面，像代码这样的结构化请求自然比开放式聊天保持更高的接受率（Abramovich等人，2026；Xia等人，2024）。在系统方面，在轻负载下验证额外token几乎是免费的。然而，在重负载下，验证具有高拒绝风险的token会占用关键的批处理容量，而这些容量本可以用于服务其他活动请求（Liu等人，2024b；Wu等人，2025）。

To address these bottlenecks, we introduce DSpark, a speculative decoding framework that unifies high-throughput parallel generation with adaptive, load-aware verification. At its core, DSpark is designed to resolve the inherent trade-offs in draft generation and verification through two complementary mechanisms.

为解决这些瓶颈，我们提出DSpark，一个将高吞吐量并行生成与自适应、负载感知的验证相结合的推测解码框架。DSpark的核心设计目标是通过两个互补机制来解决草稿生成和验证中固有的权衡问题。

- First, to overcome the lack of inter-token dependencies, DSpark adopts a semi-autoregressive architecture. It keeps the computationally expensive draft backbone fully parallel, appending only a lightweight serial output head to inject local transition information. This design preserves the drafting speed of parallel models while significantly mitigating suffix decay.

- 首先，为克服token间依赖关系的缺失，DSpark采用半自回归架构。它保持计算密集型的草稿主干完全并行，仅附加一个轻量级序列输出来注入局部转移信息。这种设计在保持并行模型草稿速度的同时，显著缓解了后缀衰减。

- Second, to resolve the system-level bottleneck, DSpark employs confidence-scheduled verification. By coupling a confidence head—which estimates per-position prefix survival probabilities—with a hardware-aware scheduler, DSpark dynamically tailors the verification length for each request. This scheduler leverages real-time engine throughput profiles to route target verification budget only toward tokens with the highest expected return.

- 其次，为解决系统级瓶颈，DSpark采用置信度调度验证。通过将估计每个位置前缀存活概率的置信度头与硬件感知调度器相结合，DSpark为每个请求动态调整验证长度。该调度器利用实时引擎吞吐量配置文件，仅将目标验证预算分配给具有最高预期回报的token。

We extensively evaluate DSpark across both controlled offline benchmarks and production-scale online deployments. On controlled offline benchmarks—spanning mathematical reasoning, code generation, and daily chat—DSpark consistently outperforms strong baselines. Specifically, across the Qwen3-4B, 8B, and 14B target models (Yang et al., 2025), it improves the macro-average accepted length over the autoregressive Eagle3 (Li et al., 2026b) by 30.9%, 26.7%, and 30.0%, and over the parallel DFlash (Chen et al., 2026) by 16.3%, 18.4%, and 18.3%, respectively. Beyond topline metrics, our fine-grained position-wise analysis reveals the distinct generation characteristics of different drafters, empirically demonstrating how DSpark successfully combines the high initial-token capacity of parallel models with the suffix coherence of autoregressive models.

我们在受控的离线基准测试和生产规模的在线部署中对DSpark进行了广泛评估。在涵盖数学推理、代码生成和日常聊天的受控离线基准测试中，DSpark始终优于强大的基线模型。具体而言，在Qwen3-4B、8B和14B目标模型上（Yang等人，2025），它相比自回归的Eagle3（Li等人，2026b）分别将宏平均接受长度提高了30.9%、26.7%和30.0%，相比并行的DFlash（Chen等人，2026）分别提高了16.3%、18.4%和18.3%。除了顶级指标外，我们的细粒度逐位置分析揭示了不同草稿模型的独特生成特征，经验性地证明了DSpark如何成功地将并行模型的高初始token容量与自回归模型的后缀连贯性相结合。

Beyond offline evaluation, we deployed DSpark within the DeepSeek-V4 (DeepSeek-AI, 2026) serving system to assess its performance under live user traffic. Compared to the prior MTP-1 production baseline (DeepSeek-AI, 2024), DSpark significantly broadens the system's operational envelope. Specifically, it consistently accelerates per-user generation speeds by 60%–85% (V4-Flash) and 57%–78% (V4-Pro) at matched aggregate throughput capacities. Furthermore, under strict Service Level Agreements (SLAs) where the baseline's capacity deteriorates severely—such as 120 TPS for Flash and 50 TPS for Pro—DSpark mitigates verification overhead to maintain robust throughput. By overcoming this performance cliff, DSpark unlocks strict interactivity tiers that were previously unattainable, effectively shifting the Pareto frontier of LLM serving.

除了离线评估，我们还将DSpark部署在DeepSeek-V4（DeepSeek-AI，2026）服务系统中，以评估其在真实用户流量下的性能。与之前的MTP-1生产基线（DeepSeek-AI，2024）相比，DSpark显著拓宽了系统的运行范围。具体而言，在相同的总吞吐量容量下，它始终将每用户生成速度提升60%–85%（V4-Flash）和57%–78%（V4-Pro）。此外，在基线容量严重恶化的严格服务级别协议（SLA）下——例如Flash的120 TPS和Pro的50 TPS——DSpark通过减少验证开销来维持稳定的吞吐量。通过克服这一性能悬崖，DSpark解锁了以前无法达到的严格交互性层级，有效地推动了LLM服务的帕累托前沿。

To foster collective advancement within the open-source community, we are making our artifacts publicly available. Specifically, we release the trained DSpark checkpoints for both the DeepSeek-V4-Flash (preview) and DeepSeek-V4-Pro (preview) models. Furthermore, we open-source DeepSpec, an algorithm-driven training repository, including Eagle3, DFlash and DSpark. These artifacts are intended to support further research on efficient LLM serving.

为促进开源社区的共同进步，我们将我们的成果公开发布。具体而言，我们发布了DeepSeek-V4-Flash（预览版）和DeepSeek-V4-Pro（预览版）模型的训练后DSpark检查点。此外，我们开源了DeepSpec，一个算法驱动的训练仓库，包括Eagle3、DFlash和DSpark。这些成果旨在支持高效LLM服务的进一步研究。

2. Background

2. 背景

2.1. Speculative Decoding

2.1. 推测解码

Autoregressive language models generate one token per forward pass, making inference latency proportional to output length. Speculative decoding (Chen et al., 2023; Ge et al., 2022; Leviathan et al., 2023) accelerates the inference of a target model M_t using a lightweight draft model M_d . At each decoding cycle, the draft model proposes γ candidate tokens $x_1, \dots, x_\gamma$. The target model verifies all candidates in a single forward pass, accepting the longest prefix consistent with its own distribution.

自回归语言模型每次前向传播生成一个token，使得推理延迟与输出长度成正比。推测解码（Chen等人，2023；Ge等人，2022；Leviathan等人，2023）使用轻量级草稿模型 M_d 来加速目标模型 M_t 的推理。在每个解码周期，草稿模型提出 γ 个候选token $x_1, \dots, x_\gamma$ 。目标模型在单次前向传播中验证所有候选，接受与其自身分布一致的最长前缀。

Concretely, at each draft position k , the target model computes its own distribution p_k^t and compares it against the draft distribution p_k^d . The token x_k is accepted with probability $\min(1, p_k^t(x_k) / p_k^d(x_k))$. Verification proceeds left to right: the first rejection at position k discards all subsequent tokens $x_{k+1}, \dots, x_\gamma$, regardless of their quality.

具体而言，在每个草稿位置 k ，目标模型计算其自身的分布 p_k^t 并将其与草稿分布 p_k^d 进行比较。token x_k 以概率 $\min(1, p_k^t(x_k) / p_k^d(x_k))$ 被接受。验证从左到右进行：位置 k 的第一次拒绝会丢弃所有后续token $x_{k+1}, \dots, x_\gamma$ ，无论它们的质量如何。

Let τ denote the number of accepted tokens per cycle, and let T_{draft} and T_{verify} be the wall-clock times of the drafting and verification passes, respectively. The average latency per generated token is: $L = (T_{\text{draft}} + T_{\text{verify}}) / \tau$. (1)

设 τ 表示每个周期接受的token数， T_{draft} 和 T_{verify} 分别表示草稿和验证过程的实际时间。每个生成token的平均延迟为： $L = (T_{\text{draft}} + T_{\text{verify}}) / \tau$ 。（1）

Improving speedup therefore reduces to three levers: lowering T_{draft} (draft faster), raising τ (draft better), or reducing the effective T_{verify} (verify smarter).

因此，提高加速比归结为三个杠杆：降低 T_{draft} （更快地打草稿）、提高 τ （更好地打草稿）或降低有效 T_{verify} （更智能地验证）。

2.2. Drafter Architectures

2.2. 草稿模型架构

The design of the draft model determines how T_{draft} and τ trade off. Existing approaches fall into two categories.

草稿模型的设计决定了 T_{draft} 和 τ 之间的权衡。现有方法分为两类。

Autoregressive drafters. Autoregressive drafters generate draft tokens sequentially, conditioning each position on previously sampled tokens (DeepSeek-AI, 2024; Li et al., 2024b,c, 2026b; Zhang et al., 2025). This explicit dependency gives strong modeling capacity, but the drafting cost grows linearly with block size: $T_{\text{draft}} \propto \gamma$, which forces autoregressive drafters to use small γ and shallow architectures to keep T_{draft} low. To compensate for the short block, tree-based verification (Miao et al., 2024) expands candidates into a tree and verifies multiple paths via tree attention, but the large number of verification tokens reduces overall serving throughput.

自回归草稿模型。自回归草稿模型按顺序生成草稿token，每个位置都基于先前采样的token进行条件生成（DeepSeek-AI, 2024；Li等人，2024b,c, 2026b；Zhang等人，2025）。这种显式依赖提供了强大的建模能力，但草稿成本随块大小线性增长： $T_{\text{draft}} \propto \gamma$ ，这迫使自回归草稿模型使用小的 γ 和浅层架构来保持低 T_{draft} 。为了弥补短块的不足，基于树的验证（Miao等人，2024）将候选扩展为树并通过树注意力验证多条路径，但大量的验证token降低了整体服务吞吐量。

Parallel drafters. Parallel drafters produce all γ draft tokens in a single forward pass, making T_{draft} nearly independent of the block size (Cai et al., 2024; Chen et al., 2026; Li et al., 2025a; Liu et al., 2026a; Sandler et al.,

2026). This allows substantially larger blocks (e.g., $\gamma=16$) without proportionally increasing latency.

并行草稿模型。并行草稿模型在单次前向传播中生成所有 γ 个草稿token，使得 T_{draft} 几乎与块大小无关（Cai等人，2024；Chen等人，2026；Li等人，2025a；Liu等人，2026a；Sandler等人，2026）。这允许使用大得多的块（例如 $\gamma=16$ ）而不会成比例地增加延迟。

Among them, DFlash (Chen et al., 2026) is a state-of-the-art parallel drafter, which conditions its draft model on rich context features extracted from the target model (KV injection). During prefill, hidden states from a set of target layers $\{l_1, \dots, l_m\}$ are concatenated and projected into the draft hidden space:

其中，DFlash（Chen等人，2026）是最先进的并行草稿模型，它将草稿模型基于从目标模型提取的丰富上下文特征（KV注入）进行条件生成。在预填充阶段，来自一组目标层 $\{l_1, \dots, l_m\}$ 的隐藏状态被连接并投影到草稿隐藏空间中：

$$H_{\text{ctx}} = \text{RMSNorm}(W_c[H^{(l_1)}; \dots; H^{(l_m)}]), \quad (2)$$

$$H_{\text{ctx}} = \text{RMSNorm}(W_c[H^{(l_1)}; \dots; H^{(l_m)}]), \quad (2)$$

where $W_c \in \mathbb{R}^{(d \times md)}$ is a shared projection. These context features are injected into every draft layer by concatenating them with the draft block representations along the sequence dimension of keys and values:

其中 $W_c \in \mathbb{R}^{(d \times md)}$ 是共享投影。这些上下文特征通过在键和值的序列维度上与草稿块表示连接，被注入到每个草稿层中：

$$K_i = [W_i^K H_{\text{ctx}}; W_i^K H_d], \quad V_i = [W_i^V H_{\text{ctx}}; W_i^V H_d]. \quad (3)$$

$$K_i = [W_i^K H_{\text{ctx}}; W_i^K H_d], \quad V_i = [W_i^V H_{\text{ctx}}; W_i^V H_d]. \quad (3)$$

All positions within a block attend bidirectionally to each other and to the injected target context.

块内的所有位置相互之间以及与注入的目标上下文之间进行双向注意力。

The draft model shares the target model's embedding layer and language modeling head (both frozen). It takes as input the embedding of an anchor token followed by γ mask token embeddings, and produces logits for all mask positions in a single forward pass. Since drafting requires only a single forward pass regardless of block size, DFlash can afford deeper architectures and larger blocks than autoregressive drafters under the same latency budget.

草稿模型共享目标模型的嵌入层和语言建模头（均为冻结状态）。它以锚定token的嵌入后跟 γ 个掩码token嵌入作为输入，并在单次前向传播中为所有掩码位置生成logits。由于无论块大小如何，打草稿只需要单次前向传播，因此在相同的延迟预算下，DFlash可以采用比自回归草稿模型更深的架构和更大的块。

3. Architecture

3. 架构

The overview of DSpark is shown in Figure 1. Recall from Equation 1 that the per-token latency of speculative decoding is $L = (T_{\text{draft}} + T_{\text{verify}}) / \tau$. Autoregressive drafters achieve high τ but pay $T_{\text{draft}} \propto \gamma$; parallel drafters collapse T_{draft} to a single pass but sacrifice τ because each position is predicted independently. Meanwhile, fixed-length verification wastes T_{verify} on low-confidence suffix tokens that are almost certain to be rejected. DSpark addresses these limitations with two complementary components:

DSpark的概述如图1所示。回顾公式1，推测解码的每token延迟为 $L = (T_{\text{draft}} + T_{\text{verify}}) / \tau$ 。自回归草稿模型实现了高 τ 但付出 $T_{\text{draft}} \propto \gamma$ 的代价；并行草稿模型将 T_{draft} 压缩为单次前向传播但牺牲了 τ ，因为每个位置是独立预测的。同时，固定长度验证在几乎肯定会被拒绝的低置信度后缀token上浪费了 T_{verify} 。DSpark通过两个互补组件解决了这些限制：

- Semi-autoregressive generation (Section 3.1). A parallel backbone handles the bulk of draft computation, which keeps T_{draft} nearly independent of γ . A lightweight sequential block then injects dependency among draft tokens, improving τ at minimal additional latency.

- 半自回归生成（第3.1节）。并行主干处理大部分草稿计算，这使得 T_{draft} 几乎与 γ 无关。然后轻量级序列块在草稿token之间注入依赖关系，以最小的额外延迟提高 τ 。

- Confidence-scheduled verification (Section 3.2). A confidence head estimates per-position acceptance probabilities, and a hardware-aware scheduler uses these estimates to prune low-confidence suffix tokens, cutting unnecessary verification compute.

- 置信度调度验证（第3.2节）。置信度头估计每个位置的接受概率，硬件感知调度器使用这些估计来剪枝低置信度后缀token，减少不必要的验证计算。

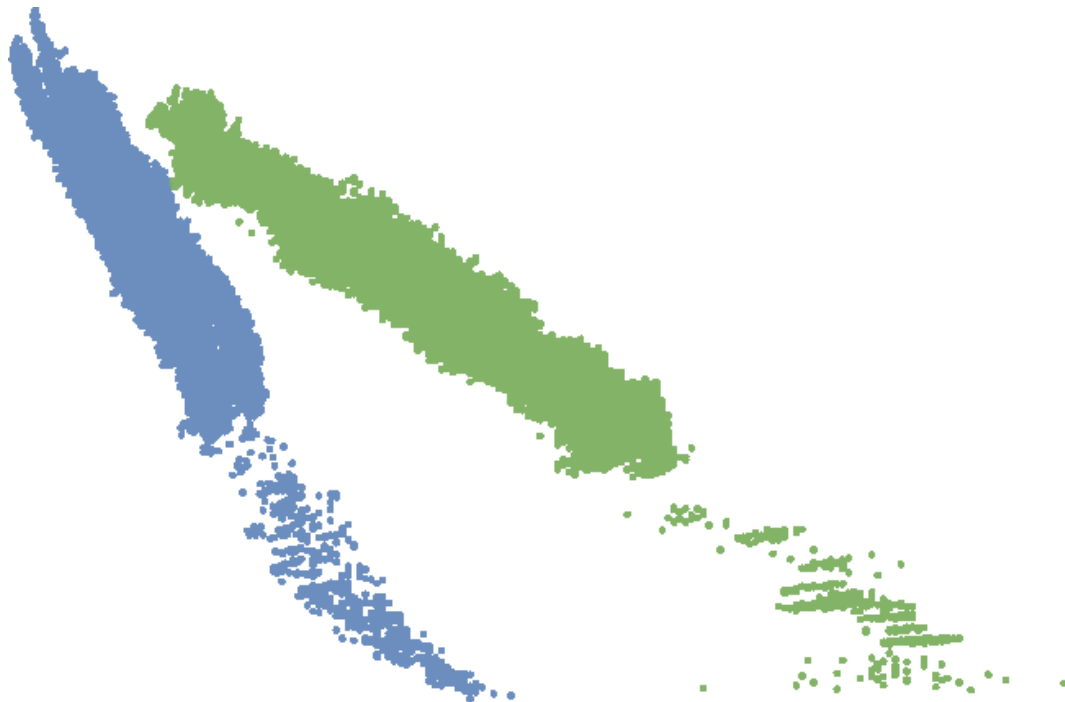


Figure 1 | The DSpark architecture and decoding cycle. (■1 | DSpark■■■■■■■■■■)

In combination, the two components let DSpark draft better and verify smarter. We detail each below.

这两个组件相结合，使DSpark能够更好地打草稿并更智能地验证。我们将在下面详细介绍每一个。

3.1. Semi-Autoregressive Generation

3.1. 半自回归生成

A parallel drafter produces all γ draft logits in one forward pass, so each prediction cannot condition on tokens sampled elsewhere in the block. When the context admits multiple plausible continuations, e.g., "of course" and "no problem", a parallel drafter may produce incoherent combinations such as "of problem" or "no course", because each position marginalizes over all possible predecessors rather than conditioning on the one actually sampled (Gu et al., 2018; Huang et al., 2022a). Acceptance rate thus decays rapidly along the block, wasting both draft and verification compute. We therefore adopt a semi-autoregressive structure that splits draft generation into two stages:

并行草稿模型在一次前向传播中生成所有 γ 个草稿logit，因此每个预测不能以块中其他位置采样的token为条件。当上下文允许多个合理的延续时，例如"of course"和"no problem"，并行草稿模型可能会产生不连贯的组合，如"of problem"或"no course"，因为每个位置对所有可能的前驱进行边缘化，而不是以实际采样的那个为条件（Gu等人，2018；Huang等人，2022a）。因此，接受率沿块快速衰减，浪费了草稿和验证计算。因此，我们采用半自回归结构，将草稿生成分为两个阶段：

Parallel stage. A parallel backbone (in our instantiation, DFlash (Chen et al., 2026)) runs a single forward pass over the entire block, producing hidden states $h_1, \dots, h_\gamma$ and base logits $U_1, \dots, U_\gamma$. We make only a minor modification to the original DFlash backbone: instead of feeding an anchor token plus γ mask tokens and predicting only the mask positions, we treat the anchor itself as the first prediction position, so γ input tokens (anchor + $\gamma-1$ masks) yield γ draft logits. This reduces draft computation while maintaining similar draft quality.

并行阶段。并行主干（在我们的实现中是DFlash（Chen等人，2026））在整个块上运行单次前向传播，产生隐藏状态 $h_1, \dots, h_\gamma$ 和基础logits $U_1, \dots, U_\gamma$ 。我们对原始DFlash主干只做了一个小修改：不是输入一个锚定token加 γ 个掩码token并只预测掩码位置，而是将锚定本身视为第一个预测位置，这样 γ 个输入token（锚定 + $\gamma-1$ 个掩码）产生 γ 个草稿logits。这减少了草稿计算，同时保持了相似的草稿质量。

Sequential stage. The sequential stage supplements the base logits with a prefix-dependent transition bias $B_k(x_0, x_{<k}, x_k)$, allowing each draft position to condition on previously sampled tokens within the block. Rather than defining a globally normalized energy model, the sequential stage induces a causal block distribution through an autoregressive factorization:

序列阶段。序列阶段用前缀依赖的转移偏置 $B_k(x_0, x_{<k}, x_k)$ 补充基础logits，允许每个草稿位置以块内先前采样的token为条件。序列阶段不是定义全局归一化的能量模型，而是通过自回归分解诱导因果块分布：

$$\begin{aligned} P(X | x_0) &= \prod_{k=1}^{\gamma} p_k(x_k | x_0, x_{<k}), \\ p_k(v | x_0, x_{<k}) &= \exp(U_k(v) + B_k(x_0, x_{<k}, v)) / \sum_{u \in V} \exp(U_k(u) + B_k(x_0, x_{<k}, u)). \quad (4) \\ P(X | x_0) &= \prod_{k=1}^{\gamma} p_k(x_k | x_0, x_{<k}), \\ p_k(v | x_0, x_{<k}) &= \exp(U_k(v) + B_k(x_0, x_{<k}, v)) / \sum_{u \in V} \exp(U_k(u) + B_k(x_0, x_{<k}, u)). \quad (4) \end{aligned}$$

Here, x_0 denotes the anchor token from the previous verification cycle, U_k is the base logit vector produced by the parallel backbone at position k , and V is the vocabulary. At inference time, the sequential block samples left to right according to $p_k(\cdot | x_0, x_{<k})$. Because this sampling process is inherently sequential, the block must be computationally lightweight ($T_{\text{sequential}} \ll T_{\text{parallel}}$) so that the overall draft latency remains dominated by the parallel stage. We describe two instantiations of the sequential block below.

其中， x_0 表示来自上一个验证周期的锚定token， U_k 是并行主干在位置 k 产生的基础logit向量， V 是词表。在推理时，序列块根据 $p_k(\cdot | x_0, x_{<k})$ 从左到右采样。由于这个采样过程本质上是序列的，该块必须在计算上是轻量级的（ $T_{\text{sequential}} \ll T_{\text{parallel}}$ ），以便整体草稿延迟仍然由并行阶段主导。我们在下面描述序列块的两种实现。

• **Markov head.** The simplest instantiation restricts B_k to depend only on the immediately preceding token, reducing it to a first-order transition $B(x_{k-1}, x_k)$. In principle this is a full $V \times V$ matrix B ; we approximate it with a low-rank factorization $B = W_1 W_2$, where $W_1 \in \mathbb{R}^{(V \times r)}$ and $W_2 \in \mathbb{R}^{(r \times V)}$. Given the preceding token x_{k-1} , the transition bias for position k is:

• 马尔可夫头。最简单的实现将 B_k 限制为仅依赖于紧接在前的token，将其简化为一阶转移 $B(x_{k-1}, x_k)$ 。原则上这是一个完整的 $V \times V$ 矩阵 B ；我们用低秩分解 $B = W_1 W_2$ 来近似它，其中 $W_1 \in \mathbb{R}^{(V \times r)}$ ， $W_2 \in \mathbb{R}^{(r \times V)}$ 。给定前一个token x_{k-1} ，位置 k 的转移偏置为：

$$\begin{aligned} B(x_{k-1}, \cdot) &= W_1[x_{k-1}] W_2 \in \mathbb{R}^{V \times V}, \quad (5) \\ B(x_{k-1}, \cdot) &= W_1[x_{k-1}] W_2 \in \mathbb{R}^{V \times V}, \quad (5) \end{aligned}$$

where W_1 serves as an embedding lookup table and W_2 as a logit projection. The low-rank factorization ($r=256$ by default) keeps both storage and per-step compute small, making the sequential loop efficient even for large vocabularies. Returning to the earlier example: once position 1 samples "of", the Markov head boosts "course" and suppresses "problem" at position 2, which mitigates the cross-mode collision.

其中 W_1 用作嵌入查找表， W_2 用作logit投影。低秩分解（默认 $r=256$ ）保持了存储和每步计算的小规模，使得序列循环即使对于大词表也很高效。回到前面的例子：一旦位置1采样了"of"，马尔可夫头在位置2会增强"course"并抑制"problem"，这减轻了跨模式碰撞。

• **RNN head.** The Markov head is memoryless beyond one step—position k cannot access tokens before $x_{\{k-1\}}$. The RNN head relaxes this by maintaining a recurrent state s_k that accumulates the full prefix history within a block. At each step, the module concatenates the current state $s_{\{k-1\}} \in R^r$, the previous token embedding $W_1[x_{\{k-1\}}] \in R^r$, and the backbone hidden $h_k \in R^d$ into an input vector $z_k = [s_{\{k-1\}}; W_1[x_{\{k-1\}}]; h_k] \in R^{(2r+d)}$, then applies a single gated update:

• RNN头。马尔可夫头在一步之外是无记忆的——位置 k 无法访问 $x_{\{k-1\}}$ 之前的token。RNN头通过维护一个循环状态 s_k 来放松这一限制，该状态累积块内的完整前缀历史。在每一步，该模块将当前状态 $s_{\{k-1\}} \in R^r$ 、前一个token嵌入 $W_1[x_{\{k-1\}}] \in R^r$ 和主干隐藏状态 $h_k \in R^d$ 连接成输入向量 $z_k = [s_{\{k-1\}}; W_1[x_{\{k-1\}}]; h_k] \in R^{(2r+d)}$ ，然后应用单个门控更新：

$$\begin{aligned} s_k &= \sigma(W_g z_k) \blacksquare s_{\{k-1\}} + \dots \\ B_k(x_{\{<k\}}, \cdot) &= W_2 \blacksquare \tanh(W_o z_k), \quad (6) \\ s_k &= \sigma(W_g z_k) \odot s_{\{k-1\}} + \dots \\ B_k(x_{\{<k\}}, \cdot) &= W_2 \wedge \tanh(W_o z_k), \quad (6) \end{aligned}$$

where $W_g, W_c, W_o \in R^{(2r+d) \times r}$ are jointly parameterized by a single linear projection that is split into gate, candidate, and output components. The state s_0 is initialized to zero.

其中 $W_g, W_c, W_o \in R^{(2r+d) \times r}$ 由单个线性投影联合参数化，该投影被分为门控、候选和输出组件。状态 s_0 初始化为零。

3.2. Confidence-Scheduled Verification

3.2. 置信度调度验证

The semi-autoregressive architecture enables DSpark to generate large draft blocks efficiently. However, producing more draft tokens does not automatically translate to higher end-to-end speedups. Indiscriminately verifying the full draft block can actually degrade overall system throughput, especially in high-concurrency scenarios (Hu et al., 2026; Liu et al., 2024c).

半自回归架构使DSpark能够高效生成大型草稿块。然而，生成更多草稿token并不会自动转化为更高的端到端加速比。不加区分地验证整个草稿块实际上可能会降低整体系统吞吐量，特别是在高并发场景下（Hu等人，2026；Liu等人，2024c）。

This performance bottleneck stems from two interacting factors. First, on the data side, draft acceptance rates inherently vary across domains: structured text like code naturally yields high acceptance, whereas open-ended chat has significantly lower acceptance (Abramovich et al., 2026; Xia et al., 2024). Second, on the system side, the actual cost of verifying an extra token depends strictly on the engine load. Under light system load, an extra verification incurs minimal penalty even if rejected. However, under high-concurrency deployments, every unnecessary verification occupies target model batch capacity that could otherwise serve other active requests (Liu et al., 2024b; Wu et al., 2025).

这个性能瓶颈源于两个相互作用的因素。首先，在数据方面，草稿接受率本质上因领域而异：像代码这样的结构化文本自然产生高接受率，而开放式聊天的接受率要低得多（Abramovich等人，2026；Xia等人，2024）。其次，在系统方面，验证额外token的实际成本严格取决于引擎负载。在轻系统负载下，即使被拒绝，额外的验证也只会产生最小的惩罚。然而，在高并发部署中，每个不必要的验证都会占用目标模型的批处理容量，而这些容量本可以用于服务其他活动请求（Liu等人，2024b；Wu等人，2025）。

Therefore, fully unlocking the potential of large draft blocks requires a unified mechanism that routes target model compute only toward tokens with a positive expected return. DSpark achieves this by coupling a confidence head (Section 3.2.1) that predicts prefix survival probabilities, with a hardware-aware prefix scheduler (Section 3.2.2) that dynamically determines the optimal verification lengths based on current system load.

因此，充分释放大型草稿块的潜力需要一个统一机制，仅将目标模型计算分配给具有正预期回报的token。DSpark通过将预测前缀存活概率的置信度头（第3.2.1节）与根据当前系统负载动态确定最佳验证长度的硬件感知前缀调度器（第3.2.2节）相结合来实现这一点。

4. Experiments

4. 实验

4.1. Experimental Setup

4.1. 实验设置

Target and draft models. We evaluate DSpark on four target models spanning different scales and model families: Qwen3-{4B, 8B, 14B} (Yang et al., 2025), and Gemma4-12B (Google DeepMind, 2026). For draft models, we compare DSpark with two representative drafters: DFlash (Chen et al., 2026), a state-of-the-art parallel drafter, and Eagle3 (Li et al., 2026b), an autoregressive drafter based on Training-Time Test (TTT). For fair comparison, we retrain all drafters in the same training framework and on the same data. We align Eagle3’s TTT horizon (7) with the block size (7) used by DFlash and DSpark, and we use the same target-model feature layers for all drafters. For the number of draft model layers, we set 1 for Eagle3 and 5 for DSpark and DFlash (Chen et al., 2026). Unless otherwise stated, DSpark denotes the Markov-head variant; we study the RNN-head variant in Section 4.3.2.

目标和草稿模型。我们在四个跨越不同规模和模型家族的目标模型上评估DSpark：Qwen3-{4B, 8B, 14B}（Yang等人，2025）和Gemma4-12B（Google DeepMind，2026）。对于草稿模型，我们将DSpark与两个代表性草稿模型进行比较：DFlash（Chen等人，2026），最先进的并行草稿模型，以及Eagle3（Li等人，2026b），基于训练时测试（TTT）的自回归草稿模型。为公平比较，我们在相同的训练框架和相同的数据上重新训练所有草稿模型。我们将Eagle3的TTT视野（7）与DFlash和DSpark使用的块大小（7）对齐，并对所有草稿模型使用相同的目标模型特征层。对于草稿模型层数，我们为Eagle3设置1层，为DSpark和DFlash设置5层（Chen等人，2026）。除非另有说明，DSpark表示马尔可夫头变体；我们在第4.3.2节研究RNN头变体。

Training data. We use Open-PerfectBlend 2, an open-sourced version of PerfectBlend (Xu et al., 2024) consisting of 1.3 million samples. It is a general-purpose instruction dataset containing chat (17.6%), math (39.4%), code (38.9%), and instruction-following data (4.1%). We only use the prompts from Open-PerfectBlend; responses are regenerated by each target model with recommended sampling parameters. Each drafter is trained for 10 epochs to ensure full convergence. For data generation and evaluation, we adopt the non-thinking mode.

训练数据。我们使用Open-PerfectBlend 2，PerfectBlend（Xu等人，2024）的开源版本，包含130万个样本。这是一个通用指令数据集，包含聊天（17.6%）、数学（39.4%）、代码（38.9%）和指令遵循数据（4.1%）。我们只使用Open-PerfectBlend中的提示；响应由每个目标模型使用推荐的采样参数重新生成。每个草稿模型训练10个epoch以确保完全收敛。对于数据生成和评估，我们采用非思考模式。

Evaluation protocol. We evaluate the performance of different algorithms on three domains:

评估协议。我们在三个领域评估不同算法的性能：

1. Mathematical Reasoning, including GSM8K (Cobbe et al., 2021), MATH500 (Lightman et al., 2024) and AIME25 (Zhang and Math-AI, 2025).

1. 数学推理，包括GSM8K（Cobbe等人，2021）、MATH500（Lightman等人，2024）和AIME25（Zhang和Math-AI，2025）。

2. Code Generation, including MBPP (Austin et al., 2021b), HumanEval (Chen et al., 2021) and Live-CodeBench (Jain et al., 2025).

2. 代码生成，包括MBPP（Austin等人，2021b）、HumanEval（Chen等人，2021）和Live-CodeBench（Jain等人，2025）。

3. Daily Chat, including MT-Bench (Zheng et al., 2023), Alpaca (Taori et al., 2023) and Arena-Hard (Li et al., 2024a, 2025b).

3. 日常聊天，包括MT-Bench（Zheng等人，2023）、Alpaca（Taori等人，2023）和Arena-Hard（Li等人，2024a, 2025b）。

For all benchmarks, we use standard speculative decoding (Chen et al., 2023; Leviathan et al., 2023) with the sampling temperature set to 1.0. We report the accepted length (τ) per decoding round. For all drafters, we use chain-based drafting.

对于所有基准测试，我们使用标准推测解码（Chen等人，2023；Leviathan等人，2023），采样温度设置为1.0。我们报告每个解码轮次的接受长度（ τ ）。对于所有草稿模型，我们使用基于链的打草稿。

4.2. Experimental Results

4.2. 实验结果

To isolate the raw draft quality from system-level scheduling policies, our offline evaluation disables the confidence scheduler, forcing all drafters to propose a fixed block of tokens. The main results, measured by the average accepted length (τ) per round, are reported in Table 1.

为了将原始草稿质量与系统级调度策略分离开，我们的离线评估禁用了置信度调度器，迫使所有草稿模型提出固定的token块。主要结果以每轮平均接受长度（ τ ）衡量，报告在表1中。

Table 1 | Main speculative decoding results. We report accepted length (τ) per decoding round (higher is better) for different target models and domains. Bold marks the best results.

Target	Drafter	Math	Code	Chat
		GSM8K / MATH / AIME25	MBPP / HumanEval / LCB	MT-Bench / Alpaca / Arena-Hard
Qwen3-4B	Eagle3	5.14 / 4.62 / 3.92	3.69 / 4.16 / 3.77	2.39 / 2.26 / 2.55
	DFlash	5.40 / 4.85 / 4.15	4.40 / 4.74 / 4.18	3.07 / 2.96 / 2.83
	DSpark	6.11 / 5.70 / 4.89	5.13 / 5.38 / 4.86	3.64 / 3.54 / 3.29
Qwen3-8B	Eagle3	5.30 / 4.77 / 3.91	3.96 / 4.33 / 4.17	2.66 / 2.54 / 2.54
	DFlash	5.33 / 4.91 / 4.07	4.36 / 4.64 / 4.39	3.11 / 2.98 / 2.81
	DSpark	6.17 / 5.78 / 5.01	5.16 / 5.52 / 5.17	3.72 / 3.58 / 3.21
Qwen3-14B	Eagle3	5.24 / 4.60 / 3.71	3.81 / 4.14 / 4.01	2.62 / 2.47 / 2.48
	DFlash	5.41 / 4.84 / 3.98	4.44 / 4.59 / 4.33	3.10 / 2.94 / 2.72
	DSpark	6.21 / 5.74 / 4.94	5.26 / 5.43 / 5.02	3.70 / 3.58 / 3.13

DSpark consistently outperforms both the autoregressive baseline (Eagle3) and the parallel baseline (DFlash) across all evaluated target models and benchmark domains. Specifically, across the Qwen3-4B, 8B, and 14B models, DSpark improves the macro-average accepted length over Eagle3 by 30.9%, 26.7%, and 30.0%, respectively. Similarly, compared to DFlash, DSpark yields relative improvements of 16.3%, 18.4%, and 18.3% across the three scales. Crucially, this advantage generalizes across model families, as demonstrated by the consistent performance gains on the Gemma4-12B target.

在所有评估的目标模型和基准测试领域中，DSpark始终优于自回归基线（Eagle3）和并行基线（DFlash）。具体而言，在Qwen3-4B、8B和14B模型上，DSpark相比Eagle3分别将宏平均接受长度提高了30.9%、26.7%和30.0%。同样，与DFlash相比，DSpark在三个规模上分别产生了16.3%、18.4%和18.3%的相对改进。至关重要，这一优势在不同模型家族中都具有普适性，正如在Gemma4-12B目标上一致的性能提升所证明的。

Beyond the average improvements, Table 1 reveals a strong domain effect: the accepted length is naturally higher on structured tasks (e.g., 5.57 on math and 5.12 on code for Qwen3-4B) than on open-ended chat (3.49). This inherent variance in data predictability means a static verification length often wastes compute on trailing tokens that are highly likely to be rejected. This directly motivates our confidence-scheduled verification, which dynamically prunes the draft block based on expected acceptance.

除了平均改进之外，表1还揭示了强烈的领域效应：结构化任务的接受长度自然更高（例如Qwen3-4B在数学上为5.57，在代码上为5.12），而开放式聊天则较低（3.49）。数据可预测性的这种内在差异意味着静态验证长度经常在极有可能被拒绝的尾部token上浪费计算。这直接推动了我们的置信度调度验证，它根据预期接受率动态剪枝草稿块。

5. Real-World Deployment of DSpark

5. DSpark的实际部署

While Section 4 establishes the algorithmic gains of DSpark on offline benchmarks, deploying it alongside large-scale models like DeepSeek-V4 (DeepSeek-AI, 2026) introduces additional system-level challenges across both training and inference. In this section, we present the end-to-end production pipeline of DSpark. We detail our scalable training mechanisms, the system-level optimizations necessary to deploy the hardware-aware prefix scheduler (Section 3.2.2), and the framework's end-to-end performance under live user traffic.

虽然第4节确立了DSpark在离线基准测试上的算法收益，但将其与DeepSeek-V4 (DeepSeek-AI, 2026) 等大规模模型一起部署会在训练和推理方面带来额外的系统级挑战。在本节中，我们介绍DSpark的端到端生产流水线。我们详细介绍了我们的可扩展训练机制、部署硬件感知前缀调度器 (第3.2.2节) 所需的系统级优化，以及该框架在真实用户流量下的端到端性能。

5.4. Performance under Live User Traffic

5.4. 真实用户流量下的性能

We evaluate DSpark-5 (configured with a maximum draft length of $\gamma=5$) against the MTP-1 (DeepSeek-AI, 2024) baseline within the production serving engines of DeepSeek-V4-Flash (preview) and DeepSeek-V4-Pro (preview). MTP-1 represents the former production setup, having been superseded by DSpark two weeks following the DeepSeek-V4-preview release. This single-token setup was historically maintained in production because deploying a static multi-token drafter (e.g., MTP-3/5) strictly degrades aggregate throughput under high concurrency due to excessive verification overhead. Therefore, comparing DSpark against this established baseline directly demonstrates its ability to safely unlock the performance potential of larger draft blocks in dynamic serving environments. In all figures, the scatter points represent raw telemetry data sampled directly from live user traffic, capturing complex, real-world request distributions, while the solid lines represent the fitted performance frontiers.

我们在DeepSeek-V4-Flash (预览版) 和DeepSeek-V4-Pro (预览版) 的生产服务引擎中，将DSpark-5 (配置最大草稿长度 $\gamma=5$) 与MTP-1 (DeepSeek-AI, 2024) 基线进行比较。MTP-1代表以前的生产设置，在DeepSeek-V4预览版发布两周后被DSpark取代。这种单token设置在历史上一直保持在生产中，因为部署静态多token草稿模型 (例如MTP-3/5) 由于过多的验证开销会在高并发下严格降低总吞吐量。因此，将DSpark与这个已建立的基线进行比较，直接证明了它在动态服务环境中安全释放更大草稿块性能潜力的能力。在所有图中，散点表示直接从真实用户流量中采样的原始遥测数据，捕获了复杂的真实世界请求分布，而实线表示拟合的性能前沿。

The Serving Pareto Frontier

服务帕累托前沿

Figure 7 illustrates the trade-off between aggregate system throughput and per-user generation speed (interactivity). To quantify DSpark's behavior under practical deployment constraints, we evaluate the system at several interactivity SLA anchors. Here, an SLA (Service Level Agreement) specifies the minimum per-user generation speed (in tokens per second) that the system must guarantee.

图7说明了总系统吞吐量与每用户生成速度 (交互性) 之间的权衡。为了量化DSpark在实际部署约束下的行为，我们在几个交互性SLA锚点评估系统。这里，SLA (服务级别协议) 指定系统必须保证的最低每用户生成速度 (以token/秒为单位)。

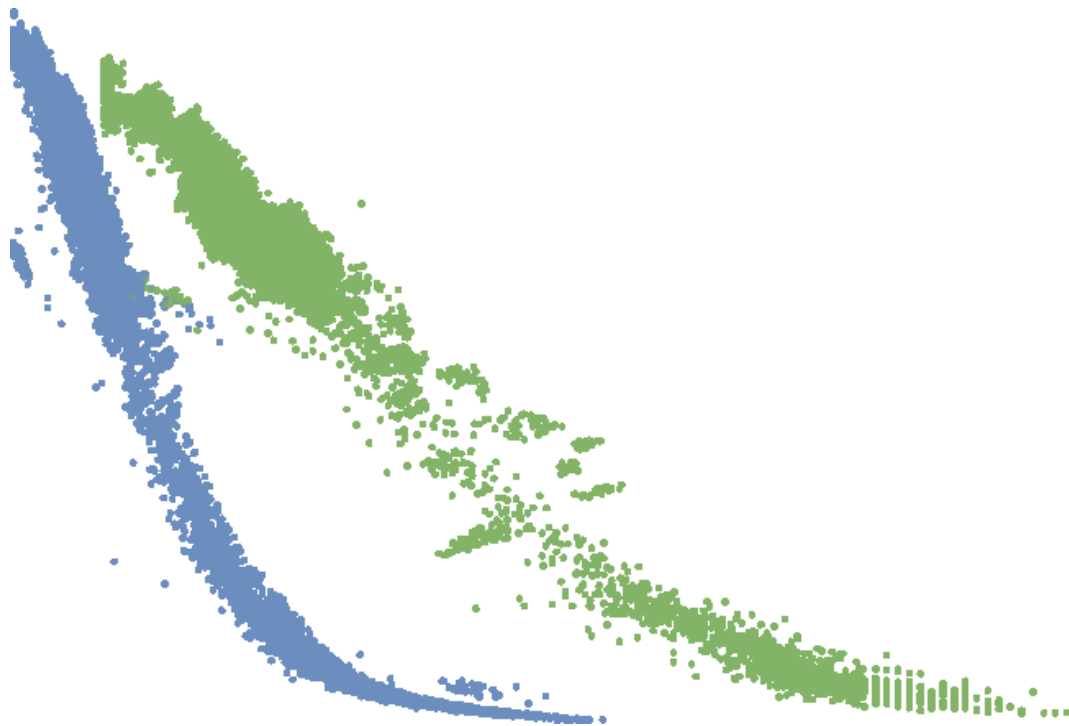


Figure 7 | Throughput vs. TPS. Aggregate output token throughput against per-request generation speed (tok/s/user) under live traffic. (■7 | ■■■■TPS■■■■■■■■■■token■■■■■■■■■■)

For the V4-Flash engine, we evaluate the system at SLA anchors of 80 and 120 tok/s/user. At the moderate 80 tok/s/user SLA, DSpark improves aggregate throughput by 51% over the MTP-1 baseline. The stricter 120 tok/s/user SLA represents a qualitatively different regime: under this constraint, the single-token MTP-1 baseline approaches its operational boundary and can sustain only a very small concurrent batch. Consequently, the relative throughput ratio at this point is numerically large, with DSpark achieving a nominal 661% higher aggregate throughput. We therefore interpret this high-SLA point primarily as evidence that DSpark extends the feasible interactivity frontier, rather than as a representative multiplicative speedup over a well-utilized baseline. At matched practical throughput levels, which provide a more stable comparison, DSpark accelerates per-user generation speeds by 60% to 85%.

对于V4-Flash引擎，我们在80和120 tok/s/user的SLA锚点评估系统。在中等的80 tok/s/user SLA下，DSpark相比MTP-1基线将总吞吐量提高了51%。更严格的120 tok/s/user SLA代表了一个质的不同状态：在这种约束下，单token MTP-1基线接近其运行边界，只能维持非常小的并发批次。因此，这一点的相对吞吐量比率在数值上很大，DSpark实现了标称661%的更高总吞吐量。因此，我们将这个高SLA点主要解释为DSpark扩展了可行的交互性前沿的证据，而不是相对于充分利用基线的代表性乘法加速。在提供更稳定比较的匹配实际吞吐量水平下，DSpark将每用户生成速度提升了60%到85%。

The V4-Pro deployment shows the same pattern. At the moderate 35 tok/s/user SLA, DSpark improves aggregate throughput by 52%. At the stricter 50 tok/s/user SLA, MTP-1 again enters a low-concurrency regime, yielding a nominal 406% relative throughput advantage for DSpark. As with V4-Flash, we treat this point as an indication that DSpark sustains useful throughput under an interactivity target that the baseline cannot efficiently support. At matched system capacities, DSpark delivers 57% to 78% faster per-user generation. Overall, these results show that DSpark shifts the observed throughput–interactivity frontier outward: it improves throughput in moderate-SLA regimes and, more importantly, preserves non-degenerate serving capacity under strict interactivity constraints.

V4-Pro部署显示了相同的模式。在中等的35 tok/s/user SLA下，DSpark将总吞吐量提高了52%。在更严格的50 tok/s/user SLA下，MTP-1再次进入低并发状态，为DSpark带来了标称406%的相对吞吐量优势。与V4-Flash一样，我们将这一点视为DSpark在基线无法有效支持的交互性目标下维持有用吞吐量的迹象。在匹配的系统容量下，DSpark提供了57%到78%更快的每用户生成速度。总体而言，这些结果表明DSpark将观察到的吞吐量-交互性前沿向外推移：它在中等SLA状态下提高了吞吐量，更重要的是，在严格的交互性约束下保持了非退化的服务容量。

6. Related Work

6. 相关工作

Speculative Decoding Algorithms

推测解码算法

Speculative decoding accelerates autoregressive generation by decoupling token proposal from verification. Building on early blockwise methods (Ge et al., 2022; Stern et al., 2018; Sun et al., 2021; Xia et al., 2023), modern approaches employ rejection sampling to exactly preserve the target model's distribution (Chen et al., 2023; Leviathan et al., 2023). Because inference speedup directly depends on the drafter's efficiency and accuracy, extensive research has focused on optimizing its architecture. Beyond using standalone small language models (Chen et al., 2023; Leviathan et al., 2023), subsequent work integrates multi-token heads or feature extrapolators directly into the target model (Ankner et al., 2024; Cai et al., 2024, 2025; DeepSeek-AI, 2024; Gloeckle et al., 2024; Li et al., 2024b,c, 2026b; Zhang et al., 2025). Other strategies include self-speculation via early exits (Elhoushi et al., 2024; Liu et al., 2024a; Xia et al., 2025; Zhang et al., 2024), dynamic vocabulary compression (Williams et al., 2026; Zhao et al., 2025b), prompt lookup (Saxena, 2023; Somasundaram et al., 2025), suffix automata (Hu et al., 2025), and retrieval (He et al., 2023; Shen et al., 2026). To remove the sequential bottleneck of drafting itself, recent methods propose parallel or blockwise generation. P-EAGLE parallelizes EAGLE-style drafting (Hui et al., 2026), while PARD, DART, and DFlash use diffusion-inspired prediction to generate entire blocks in a single forward pass (An et al., 2026; Chen et al., 2026; Liu et al., 2026a), which DDTree then extends into verifiable draft trees (Ringel and Romano, 2026). Concurrent efforts also improve DFlash: Domino (Huang et al., 2026a) introduces a CausalEncoder conceptually similar to our RNN Head, while DFlare (Zhang et al., 2026a) addresses conditioning bottlenecks via layer-wise fusion.

推测解码通过将token提议与验证解耦来加速自回归生成。在早期分块方法 (Ge等人, 2022; Stern等人, 2018; Sun等人, 2021; Xia等人, 2023) 的基础上, 现代方法采用拒绝采样来精确保持目标模型的分布 (Chen等人, 2023; Leviathan等人, 2023)。由于推理加速直接取决于草稿模型的效率和准确性, 大量研究集中在优化其架构上。除了使用独立的小语言模型 (Chen等人, 2023; Leviathan等人, 2023) 之外, 后续工作将多token头或特征外推器直接集成到目标模型中 (Ankner等人, 2024; Cai等人, 2024, 2025; DeepSeek-AI, 2024; Gloeckle等人, 2024; Li等人, 2024b,c, 2026b; Zhang等人, 2025)。其他策略包括通过早退进行自推测 (Elhoushi等人, 2024; Liu等人, 2024a; Xia等人, 2025; Zhang等人, 2024)、动态词表压缩 (Williams等人, 2026; Zhao等人, 2025b)、提示查找 (Saxena, 2023; Somasundaram等人, 2025)、后缀自动机 (Hu等人, 2025) 和检索 (He等人, 2023; Shen等人, 2026)。为了消除打草稿本身的序列瓶颈, 最近的方法提出了并行或分块生成。P-EAGLE并行化EAGLE风格的打草稿 (Hui等人, 2026), 而PARD、DART和DFlash使用受扩散启发的预测在单次前向传播中生成整个块 (An等人, 2026; Chen等人, 2026; Liu等人, 2026a), DDTree随后将其扩展为可验证的草稿树 (Ringel和Romano, 2026)。同时进行的工作也改进了DFlash: Domino (Huang等人, 2026a) 引入了概念上类似于我们RNN头的CausalEncoder, 而DFlare (Zhang等人, 2026a) 通过层级融合解决了条件瓶颈。

System-Aware Scheduling for Speculative Decoding

推测解码的系统感知调度

Beyond drafter architecture, another line of work focuses on determining the optimal number of speculative tokens to generate or verify in each round. To this end, various approaches adapt draft lengths on the fly using confidence heuristics (Du et al., 2024; Li et al., 2024b; Liu et al., 2026c; Mamou et al., 2024; Wen and Feng, 2026), learned acceptance predictors (Huang et al., 2024; Zacks917, 2026), or bandit-style policies (Liu et al., 2026b). Furthermore, recognizing speculative decoding as inherently a system-level scheduling problem, recent works optimize overall goodput and latency by adjusting speculation budgets according to real-time system load and request priority (AngelSlim Team, 2026; Hu et al., 2026; Huang et al., 2026b; Li et al., 2026a; Liu et al., 2024c; Miao et al., 2024; Sadhukhan et al., 2025; Wu et al., 2025).

除了草稿模型架构之外, 另一项工作集中在确定每轮生成或验证的最佳推测token数量。为此, 各种方法使用置信度启发式 (Du等人, 2024; Li等人, 2024b; Liu等人, 2026c; Mamou等人, 2024; Wen和Feng, 2026)、学习的接受预测器 (Huang等人, 2024; Zacks917, 2026) 或老虎机风格策略 (Liu等人, 2026b) 动态调整草稿长度。此外, 认识到推测解码本质上是一个系统级调度问题, 最近的工作通过根据实时系统负载和请求优先级调整推测预算来优化整体有效吞吐量和延迟 (AngelSlim团队, 2026; Hu等人, 2026; Huang等人, 2026b; Li等人, 2026a; Liu等人, 2024c; Miao等人, 2024; Sadhukhan等人, 2025; Wu等人, 2025)。

7. Conclusion

7. 结论

In this paper, we present DSpark, a speculative decoding framework designed to overcome the structural and system-level bottlenecks of large language model inference in high-concurrency production environments. Algorithmically, DSpark introduces a semi-autoregressive generation paradigm—coupling a computationally heavy parallel backbone with a lightweight sequential head—to mitigate the rapid suffix decay of independent parallel drafters. At the system level, we formulate verification length selection as a global throughput maximization problem, employing a hardware-aware prefix scheduler that dynamically tailors the target model's verification budget based on calibrated survival probabilities and real-time engine load. Extensive offline evaluations demonstrate that DSpark substantially outperforms state-of-the-art autoregressive and parallel baselines across diverse domains. Furthermore, its real-world deployment within the DeepSeek-V4 validates its practical value in production serving: by intelligently managing verification overhead, DSpark sustains robust concurrency under heavy load, consistently accelerates per-user generation speeds, and effectively shifts the Pareto frontier of LLM serving outward.

在本文中，我们提出了DSpark，一个推测解码框架，旨在克服高并发生产环境中大语言模型推理的结构和系统级瓶颈。在算法层面，DSpark引入了半自回归生成范式——将计算密集型的并行主干与轻量级序列头相结合——以缓解独立并行草稿模型的快速后缀衰减。在系统层面，我们将验证长度选择公式化为全局吞吐量最大化问题，采用硬件感知前缀调度器，根据校准的存活概率和实时引擎负载动态调整目标模型的验证预算。广泛的离线评估表明，DSpark在不同领域中显著优于最先进的自回归和并行基线。此外，其在DeepSeek-V4中的实际部署验证了其在生产服务中的实用价值：通过智能管理验证开销，DSpark在重负载下维持了强大的并发性，持续加快了每用户生成速度，并有效地将LLM服务的帕累托前沿向外推移。

References

参考文献

(参考文献保持英文原文，不进行翻译)

[1] T. Abramovich, M. Ashkenazi, I. Putterman, B. Chislett, T. Mitra, B. D. Rouhani, R. Zilberstein, and Y. Geifman. Speed-bench: A unified and diverse benchmark for speculative decoding. arXiv preprint arXiv:2604.09557, 2026.

[2] Z. An, H. Bai, Z. Liu, D. Li, and E. Barsoum. PARD: Accelerating LLM inference with low-cost PARallel draft model adaptation. In The Fourteenth International Conference on Learning Representations, 2026.

[3] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. Accelerating large language model decoding with speculative sampling. arXiv preprint arXiv:2302.01318, 2023.

[4] J. Chen, Y. Liang, and Z. Liu. Dflash: Block diffusion for flash speculative decoding. arXiv preprint arXiv:2602.06036, 2026.

[5] DeepSeek-AI. Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437, 2024.

[6] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026.

[7] J. Gu, J. Bradbury, C. Xiong, V. O. Li, and R. Socher. Non-autoregressive neural machine translation. In International Conference on Learning Representations, 2018.

[8] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In Proceedings of the 40th International Conference on Machine Learning, 2023.

[9] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In The Thirty-ninth Annual Conference on Neural Information Processing Systems, 2026b.

[10] X. Liu, C. Daniel, L. Hu, W. Kwon, Z. Li, X. Mo, A. Cheung, Z. Deng, I. Stoica, and H. Zhang. Optimizing speculative decoding for serving large language models using goodput. arXiv e-prints, 2024b.

... (████████████████████)

Appendices

附录

A. Counterexample: Selection Bias Without Early-Stopping

A. 反例：没有早停的选择偏差

We provide a simple counterexample to illustrate how an offline global search, i.e., operating without the break condition in Algorithm 1, violates the non-anticipating property required by lossless speculative decoding. Formally, the admission event for the k -th draft token, $\ell_r \geq k$, must be determined by scheduler-visible information available before the token $x_{\{r,k\}}$ is sampled. It must not depend on the realization of $x_{\{r,k\}}$ itself.

我们提供一个简单的反例来说明离线全局搜索（即在算法1中没有break条件的操作）如何违反无损推测解码所需的非预期属性。形式上，第 k 个草稿token的准入事件 $\ell_r \geq k$ 必须由在采样token $x_{\{r,k\}}$ 之前调度器可见的信息来确定。它不能依赖于 $x_{\{r,k\}}$ 本身的实现。

Consider a scenario with a single request ($R=1$) and maximum draft length ($\gamma=2$). Suppose the pre-token confidence for the first position is $a_1 = 0.8$, and the profiled capacity curve is $\text{SPS}(1) = 1.0$, $\text{SPS}(2) = 0.5$, $\text{SPS}(3) = 0.45$.

考虑一个单请求（ $R=1$ ）和最大草稿长度（ $\gamma=2$ ）的场景。假设第一个位置的token前置信度为 $a_1 = 0.8$ ，配置的容量曲线为 $\text{SPS}(1) = 1.0$ ， $\text{SPS}(2) = 0.5$ ， $\text{SPS}(3) = 0.45$ 。

The expected throughputs for verifying 0 and 1 draft tokens are $\Theta_0 = 1 \cdot \text{SPS}(1) = 1.0$, $\Theta_1 = (1 + 0.8) \cdot \text{SPS}(2) = 0.9$.

验证0个和1个草稿token的预期吞吐量为 $\Theta_0 = 1 \cdot \text{SPS}(1) = 1.0$ ， $\Theta_1 = (1 + 0.8) \cdot \text{SPS}(2) = 0.9$ 。

Without early-stopping, the scheduler proceeds to evaluate Θ_2 before committing any admission decisions. Because the Markov confidence head uses the previously sampled token, the next confidence score c_2 explicitly depends on the realization of x_1 . Consequently, the second-prefix survival probability $a_2 = a_1 c_2$ also depends on x_1 .

如果没有早停，调度器会在提交任何准入决定之前继续评估 Θ_2 。因为马尔可夫置信度头使用先前采样的token，所以下一个置信度分数 c_2 明确依赖于 x_1 的实现。因此，第二个前缀存活概率 $a_2 = a_1 c_2$ 也依赖于 x_1 。

Thus, the admission of the first draft token dynamically depends on the value of the first draft token itself. This retrospective dependence introduces selection bias: the scheduler favors tokens that lead to highly confident continuations, even though the admission decision for x_1 should have been made before observing x_1 .

因此，第一个草稿token的准入动态地取决于第一个草稿token本身的值。这种回顾性依赖引入了选择偏差：调度器偏爱导致高置信度延续的token，尽管 x_1 的准入决定应该在观察 x_1 之前做出。

The early-stopping mechanism prevents this issue in the causal greedy scheduler. Since $\Theta_1 < \Theta_0$, the scheduler halts immediately and returns $\ell=0$ before evaluating any continuation-dependent quantity such as c_2 . The admission decision for the first position therefore depends only on pre-token information and cannot be biased.

早停机制在因果贪心调度器中防止了这个问题。由于 $\Theta_1 < \Theta_0$ ，调度器立即停止并在评估任何依赖于延续的量（如 c_2 ）之前返回 $\ell=0$ 。因此，第一个位置的准入决定仅取决于token前信息，不会产生偏差。

--- 文档结束 ---